`WebApplication.Services`. Any scoped or transient services resolved from the DI container exposed as `IServiceScope.ServiceProvider` are disposed of automatically when you dispose of the `IServiceScope` object.

The following listing shows how you can resolve a scoped service in Program.cs using the pattern in figure 9.13. This pattern ensures that the scoped `DataContext` object is disposed of correctly before the call to `app.Run()`.

Listing 9.14 Resolving a scoped service using `IServiceScope` in Program.cs

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<DataContext>();                          ❶

WebApplication app = builder.Build();

await using (var scope = app.Services.CreateAsyncScope())           ❷
{
    var dbContext =                                                 ❸
        scope.ServiceProvider.GetRequiredService<DataContext>();    ❸
    Console.WriteLine($"Retrieved scope: {dbContext.RowCount}");
}                                                                   ❹

app.Run();
```

❶ DataContext is registered as scoped, so it shouldn't be resolved directly from app.Services.

❷ Creates an IServiceScope

❸ Resolves the scoped service from the scoped container

❹ When the IServiceScope is disposed, all resolved services are also disposed.

This example uses the async form `CreateAsyncScope()` instead of `CreateScope()`, which you generally should favor whenever possible. `CreateAsyncScope` was introduced in .NET 6 to fix an edge case